Find Power of 2

Company: JP Morgan
Round: Offline
Year: 2022
Difficulty: Medium
Topic: Bit Manipulation, Strings, Lexicographic Ordering
Source: https://www.desiqna.in/7739/jp-morgan-interview-experiences-sde-set-4

Problem Description

Given an integer N, determine whether it is a **power of 2**.

- If N is a power of 2, return its exponent.
- Otherwise, return -1.

Examples

For:
N = 8
Since:
$8 = 2^3$
Output:
3
For:
N = 5
5 is not a power of 2, so:
-1

Input Format

The first line contains an integer N, representing the given number.

Output Format

- Print the exponent X if N is a power of 2, such that:

  N = 2^X

- If N is not a power of 2, print -1.

Sample Input

5

Sample Output

-1

Explanation

Suppose the input is:

8

We need to check whether 8 is a power of 2.

We know:

$2^0 = 1$

$2^1 = 2$

$2^2 = 4$

$2^3 = 8$

Therefore:

$8 = 2^3$

So the required exponent is:

3

Hence the output is:

3

If the input is:

5

There is no integer x such that:

$2^x = 5$

Therefore, the output is:

-1

In simple terms: Find whether the given number can be written as 2^x. If yes, print x; otherwise, print -1.

Approach to Solve This Problem:

1. Start with the given number N.
2. Check whether N is divisible by 2.
3. Keep dividing N by 2 and increase a counter each time.
4. If we finally reach 1, then N is a power of 2.
5. The counter gives the required exponent.
6. If N cannot be reduced to 1, return -1.

Java Solution:

```java
import java.util.*;
public class Main {
    public static int powerOfTwo(int n) {
        if (n <= 0) {
            return -1;
        }
        int count = 0;
        while (n % 2 == 0) {
            n = n / 2;
            count++;
        }
        if (n == 1) {
            return count;
        }
        return -1;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        System.out.println(powerOfTwo(n));
        sc.close();
    }
}
```

Solution:

The main idea is to check whether the given number can be written in the form:

N = 2^X

1. Start with count = 0.
2. Repeatedly divide N by 2 while it is completely divisible by 2.
3. Increase count after every division.
4. If the final value becomes 1, then N is a power of 2, and count is the answer.
5. Otherwise, N is not a power of 2, so return -1.

Complexity Analysis:

Time Complexity: O(log N)

We repeatedly divide N by 2. The number of divisions is approximately $\log_2(N)$.

Space Complexity: O(1)

- We use only a few variables (n and count), so no extra memory is required.